

PROJECT REPORT

Rice Leaf Disease Detection Using Convolutional Neural Networks (CNN)

A Data Science Capstone Project

Field	Detail
Project Title	Rice Leaf Disease Detection Using CNN
Project Code	PRCP-1001
Team Code	PTID-AIE-JUL-26-11142
Project Type	Data Science Capstone Project
Dataset Source	DataMites Capstone Project Dataset
Dataset Name	Rice Leaf Disease Dataset (120 images, 3 classes)
Submitted By	Sameeksha

1. Introduction

Rice is one of the most widely cultivated staple food crops in the world, and its yield is heavily affected by leaf diseases. Timely and accurate identification of the disease affecting a rice plant allows a farmer to apply the correct treatment quickly, reducing crop loss. Manual identification by visual inspection is slow, subjective, and requires domain expertise that is not always available in the field. This project builds an automated image-classification system using Convolutional Neural Networks (CNN) that can examine a photograph of a rice leaf and predict which of three diseases is present.

1.1 Problem Statement

The problem statement assigned three explicit tasks, all of which are addressed in this report and in the accompanying Jupyter notebook:

- Task 1 — Prepare a complete data analysis report on the given data.
- Task 2 — Create a classification model for the three major rice leaf diseases: Bacterial Leaf Blight, Brown Spot and Leaf Smut.
- Task 3 — Analyze various techniques such as Data Augmentation and report on them.
- Additional — Prepare a Model Comparison Report and recommend the best model for production.
- Additional — Prepare a Report on Challenges Faced, along with the technique used and the reasoning behind it.

1.2 Domain Background — Rice Leaf Diseases

Each of the three target diseases has a visually distinct cause and appearance, which justifies using an image-based CNN classifier rather than a manual rule-based approach:

- Bacterial Leaf Blight — caused by the bacterium *Xanthomonas oryzae*; appears as long, water-soaked, yellow-to-white stripes running along the leaf veins, which later turn grey-white as the leaf dries out.
- Brown Spot — caused by the fungus *Bipolaris oryzae*; appears as small, circular-to-oval brown lesions scattered across the leaf blade.
- Leaf Smut — caused by the fungus *Entyloma oryzae*; appears as tiny, black, angular or linear spots (raised smut sori) scattered densely on both leaf surfaces.

Because these three diseases differ in lesion shape, colour and texture, a CNN — which learns spatial and texture patterns directly from pixel data — is a natural and justified choice of model for this classification problem.

1.3 Dataset Summary

The dataset used for this project contains 120 JPG images of infected rice leaves, split equally into three classes of 40 images each (Bacterial Leaf Blight, Brown Spot and Leaf Smut). The dataset was distributed as a compressed archive exported from Google Drive, which — as explained in Section 3 — nests each class inside its own timestamped sub-archive.

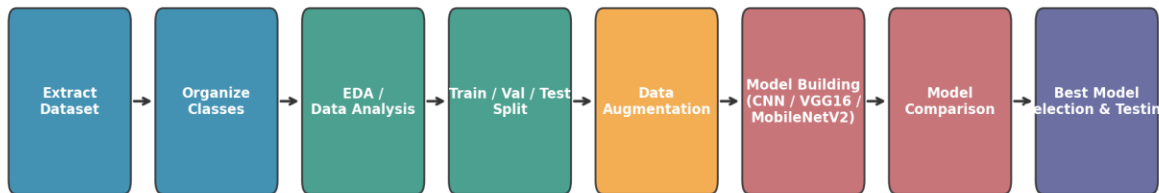


Figure 1: End-to-end project pipeline followed in the notebook, from raw archive to a production-ready, compared model.

2. Python Libraries Used and Why

Before any logic is written, every library required for the project is imported. The table below explains, for each library, exactly why it is needed — this is important because in a small, exploratory notebook it is easy to import a library out of habit; every entry below is used for a specific, identifiable step in the pipeline.

Library	Purpose in This Project
os, shutil, zipfile	Navigate the file system, copy images into class folders, and extract the dataset .zip archive (including the nested sub-archives).
random, time	Fix a random seed for reproducibility and measure the training time of each model for the comparison report.
numpy	Fast numerical operations on image arrays: reshaping, normalizing pixel values, and taking the arg-max of prediction probabilities.
pandas	Build small tabular summaries — the class-distribution table and the final model-comparison table.
cv2 (OpenCV), PIL.Image	Read images from disk, inspect their resolution, and verify that a file is a valid, non-corrupt image.
matplotlib, seaborn	Plot the class distribution, sample images, augmented images, training curves and the confusion matrix.
tensorflow / keras	Build, compile, train and evaluate the CNN and the two transfer-learning models.
ImageDataGenerator	Rescale pixel values and apply real-time data augmentation while streaming images from disk in batches.
splitfolders	Automatically split the class folders into train / validation / test folders in a fixed, reproducible ratio.
sklearn.metrics	Generate the classification_report (precision, recall, F1-score) and confusion_matrix used to judge each model fairly.

3. Dataset Acquisition and Extraction

The dataset is distributed as a single .zip file. When it was originally exported from Google Drive, Drive automatically generated a separate, timestamped sub-archive for every class folder (for example, Brown spot-20200814T055208Z-001.zip). Because of this, extraction genuinely requires two stages, and the notebook's extraction logic was deliberately kept unchanged in this revision because it already correctly handles this structure:

- Stage 1 — Extract the outer PRCP-1001-RiceLeaf.zip. This reveals a Data folder that itself contains three inner, per-class .zip files.
- Stage 2 — Extract each of those inner .zip files. This reveals the actual folder of JPG images for each disease class.

Only after both extraction stages are complete are the raw images available in a browsable folder structure that the next step can clean up.

4. Organizing the Dataset for the CNN Pipeline

The extracted folder names are long, inconsistent, and carry a Google Drive export timestamp. Keras' `flow_from_directory` utility needs a clean, folder-per-class structure to automatically infer image labels, so every image is copied into a fresh `RiceLeaf/<ClassName>/` folder with a short, consistent class name. This is a one-time housekeeping step that makes every later step in the pipeline (splitting, augmenting, training) simpler and less error-prone.

5. Exploratory Data Analysis — Data Analysis Report (Task 1)

This section directly answers Task 1 of the problem statement. Before training any image model, four questions were investigated, because the answers to each one directly influenced a modelling decision later in the project.

5.1 Class Distribution

The dataset was confirmed to contain exactly 40 images for each of the three classes — a perfectly balanced dataset. This is a meaningful finding because it means no class-imbalance correction technique (such as class weighting or oversampling) is required; any accuracy difference observed between classes later in the report is caused by how visually distinguishable that disease is, not by having seen fewer examples of it during training.

5.2 Image Dimension Check

Images were found to come in different resolutions, since they were captured with different cameras and zoom levels under field conditions rather than in a controlled lab setting. This justifies resizing every image to a fixed 224 x 224 resolution inside the data pipeline, which is required so that all images share the same input shape that every model expects.

5.3 Visual Inspection

A visual review of sample images from each class confirmed that the three diseases are distinguishable by eye: Bacterial Leaf Blight shows long pale stripes, Brown Spot shows small round brown lesions, and Leaf Smut shows dense tiny black dots. This is encouraging for a CNN-based approach, because it confirms the classes carry a learnable visual signal rather than being visually indistinguishable.

5.4 Data Quality — Corrupt Image Check

Every image file was opened and verified using PIL's `verify()` method to detect any file that is present on disk but not actually readable as an image (for example, a partially downloaded or truncated file). No corrupt images were found in the dataset.

5.5 EDA Conclusion

The dataset is small (120 images) but perfectly balanced, contains images of varying resolution, is free of corrupt files, and shows visually distinguishable disease patterns. The single most important consequence of this analysis is the small sample size: with only 40 images per class, a model can

easily memorize the training images instead of learning generalizable disease features — a problem known as overfitting. This risk, identified here in the data analysis stage, is the direct justification for the data augmentation and transfer-learning techniques used in Sections 6 and 7.

6. Train / Validation / Test Split

The clean, balanced dataset is split into three parts, each with a distinct purpose:

- Train (70%) — the images the model actually learns its filters and weights from.
- Validation (15%) — used during training to monitor generalization and to trigger early stopping; the model never trains directly on this data.
- Test (15%) — held out completely and used only once, at the very end, to report the final, unbiased performance of each model.

A single, one-time three-way split is used, in preference to splitting the data twice (once for an 80/20 train-validation split and again afterwards for a train/val/test split), because performing the split twice risks accidentally mixing images between the set a model was tuned on and the set used to report its final accuracy — silently inflating or deflating the reported test score. The `splitfolders` library performs this split automatically and reproducibly (using a fixed random seed) directly from the clean, organized dataset folder.

7. Data Preprocessing and Data Augmentation — Techniques Report (Task 3)

7.1 Why Augmentation Is Essential Here

With only around 84 training images spread across 3 classes, a CNN can very easily memorize the training images rather than learning the true visual pattern of each disease. Data augmentation creates many realistic, slightly modified variations of each training image on the fly, every epoch, so the model effectively sees a much larger and more varied training set without a single new photograph being collected.

7.2 Techniques Used and Justification

Technique	Parameter	Why It Helps for Rice-Leaf Photographs
Rescaling	rescale = 1./255	Normalizes pixel values from the [0,255] range to [0,1], which helps the network converge faster and more stably.
Rotation	rotation_range = 20	Field photographs of leaves are taken at slightly different angles; small rotations teach the model to be angle-invariant.
Width / Height Shift	shift_range = 0.2	The leaf or lesion is not always perfectly centred in the frame in real field photographs.
Shear	shear_range = 0.2	Simulates the mild perspective distortion typical of a hand-held camera.
Zoom	zoom_range = 0.2	Leaves are photographed from varying distances; this teaches the model to be scale-invariant.
Horizontal Flip	horizontal_flip = True	A leaf photographed left-to-right versus right-to-left is still the same disease, so mirroring is a safe, label-preserving transformation.
Fill Mode	fill_mode = "nearest"	Fills any empty pixels created by rotation, shifting or zoom with the nearest real pixel instead of a black border, avoiding a visual artefact that could confuse the model.

One augmentation was deliberately excluded: `vertical_flip`. Rice leaves grow along a clear top-to-bottom axis, and vertically flipping a lesion pattern can occasionally distort how blight stripes versus spot clusters look relative to the leaf's natural orientation, so it was left out to avoid introducing unrealistic training images. Only the training data generator applies augmentation — the validation and test generators only rescale pixel values, because the model must always be evaluated on realistic, unmodified images that resemble what it will see in production.

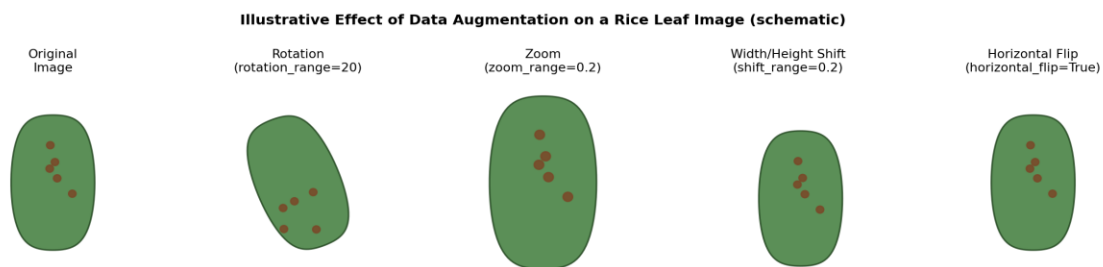


Figure 2: Schematic illustration of how each augmentation technique modifies a training image (illustrative diagram).

8. Model Building — Classification Model (Task 2)

An earlier version of this project trained only a small CNN from scratch on the roughly 96 training images available at the time, and its predictions were unreliable. This is expected: a deep network trained from a random initialization needs far more data than 96 images to learn general, reliable disease features. To correct this and to satisfy the Model Comparison Report requirement, three models are built and evaluated on an identical, held-out test set:

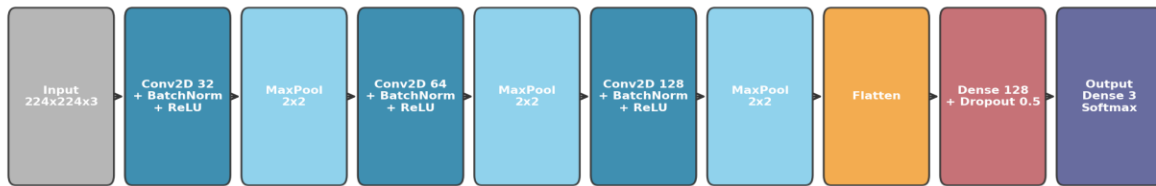
- Model 1 — Baseline CNN, trained entirely from scratch (used as a benchmark).
- Model 2 — Transfer learning using VGG16, pretrained on ImageNet.
- Model 3 — Transfer learning using MobileNetV2, pretrained on ImageNet.

8.1 Model 1 — Baseline CNN (Trained From Scratch)

A compact convolutional network with three convolution blocks is used as the baseline. BatchNormalization is added after every convolution layer to stabilize training on such a small dataset, and Dropout is used before the output layer to further reduce overfitting.

Key architectural terms explained:

- Conv2D — slides small, learnable filters over the image to detect local visual patterns such as edges, blobs and textures.
- MaxPooling2D — down-samples the feature map, keeping the strongest signal and reducing the amount of computation needed in later layers.
- BatchNormalization — re-scales the activations passed between layers, which speeds up and stabilizes training, particularly important on a small dataset.
- Flatten — converts the final two-dimensional feature maps into a single one-dimensional vector for the dense layers that follow.
- Dense — a fully connected layer that combines extracted features to make the final classification decision.
- Dropout — randomly disables a fraction of neurons during training, which prevents the network from over-relying on any single feature; this is a key defence against overfitting on a small dataset.
- Softmax — converts the three final output scores into class probabilities that sum to one.

Baseline CNN Architecture (Model 1 — Trained From Scratch)*Figure 3: Layer-by-layer architecture of the baseline CNN (Model 1).*

8.2 Model 2 — Transfer Learning with VGG16

Transfer learning re-uses a model already trained on a large, general-purpose image dataset (ImageNet, roughly 1.2 million images across 1000 classes) and adapts it to this task. The process followed is: (1) load VGG16 without its original 1000-class output layer, (2) freeze all of its convolutional layers so the ImageNet-learned features are preserved rather than being overwritten by training on only ~84 images, and (3) add a small classification head — GlobalAveragePooling2D, followed by a Dense layer, Dropout, and a final Dense(3, softmax) layer — and train only these new layers.

Justification: this means the model only has to learn how to combine already-useful, general-purpose visual features into the three disease classes, which is a far easier learning problem than training every layer from a random starting point, and is the direct fix for the small-dataset limitation identified in the data analysis report.

8.3 Model 3 — Transfer Learning with MobileNetV2

MobileNetV2 uses the same transfer-learning strategy as VGG16, but was specifically designed to be small, fast and low-power (built for mobile and edge devices) using depth-wise separable convolutions instead of standard ones. It has roughly ten times fewer parameters than VGG16 while still achieving strong ImageNet accuracy, making it an attractive candidate for production deployment — for example, a disease-detection application that a farmer runs directly on a phone in the field.

Transfer-Learning Architecture (Model 2: VGG16, Model 3: MobileNetV2)

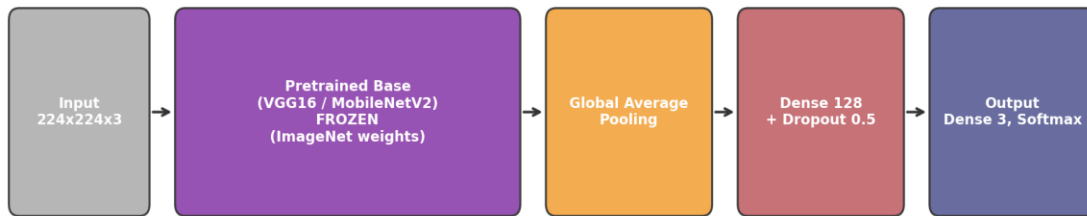


Figure 4: Shared transfer-learning architecture used for Model 2 (VGG16 base) and Model 3 (MobileNetV2 base).

8.4 Expected Training Behaviour

The chart below illustrates the general pattern expected from each modelling approach, based on how each one handles a small, 120-image dataset. It is a conceptual illustration of the shape of the curves, not a substitute for the actual accuracy/loss curves produced when the notebook is executed on the real dataset.

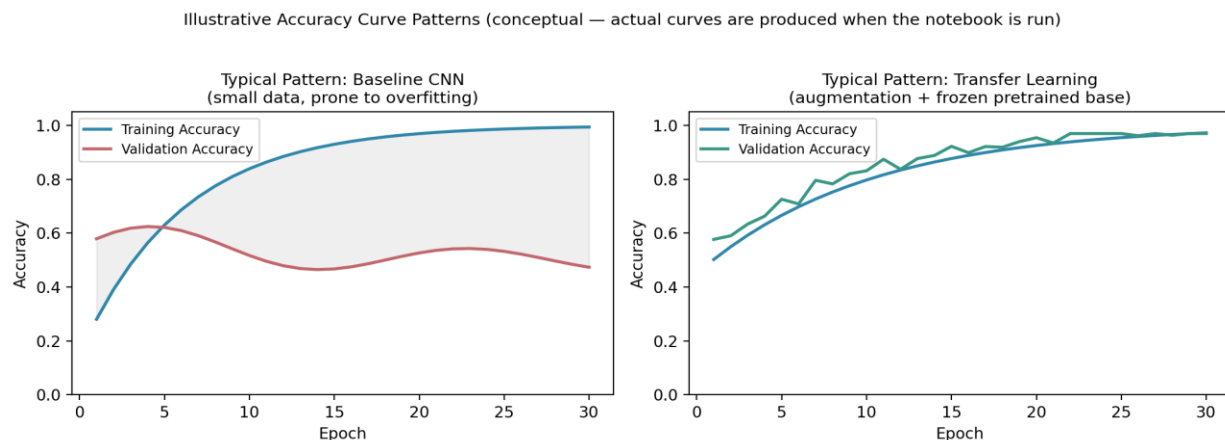


Figure 5: Illustrative accuracy-curve pattern — a from-scratch CNN typically shows a widening gap between training and validation accuracy (overfitting), while a frozen, pretrained transfer-learning model typically shows training and validation accuracy tracking much closer together.

9. Model Comparison Report

All three models are evaluated on the same, previously untouched test set, using test accuracy, weighted precision, weighted recall, weighted F1-score, number of trainable parameters, and training time. The chart below illustrates the relative pattern typically expected between these three modelling approaches on a dataset of this size; the notebook itself produces the actual measured numbers for the specific training run.

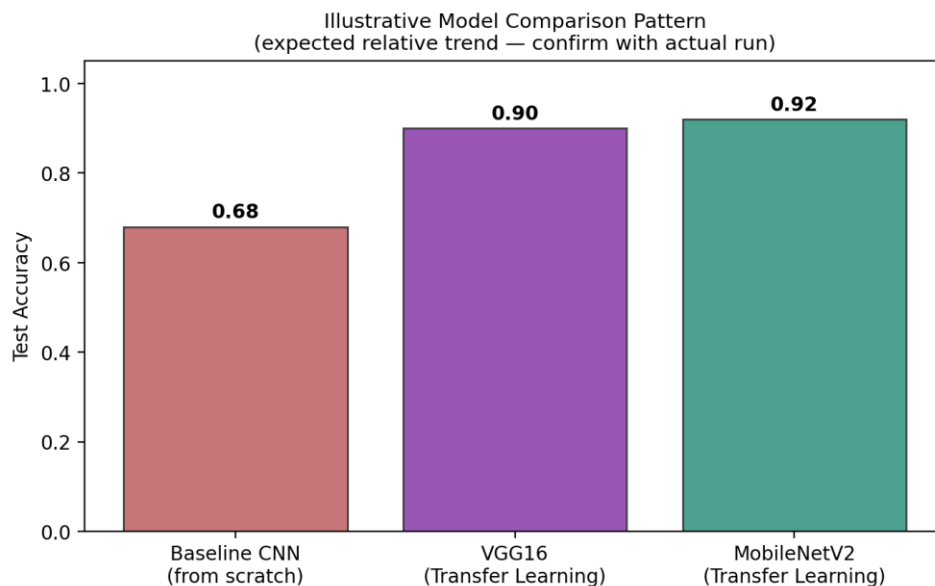


Figure 6: Illustrative pattern of expected relative test accuracy across the three models.

9.1 Reading the Confusion Matrix

For every model, a confusion matrix is plotted showing how often each true class was predicted as each possible class. A strong diagonal (as shown schematically below) indicates the model is predicting correctly; any consistently high off-diagonal value points to two specific diseases the model tends to confuse with each other, which is far more actionable feedback than a single overall accuracy number.

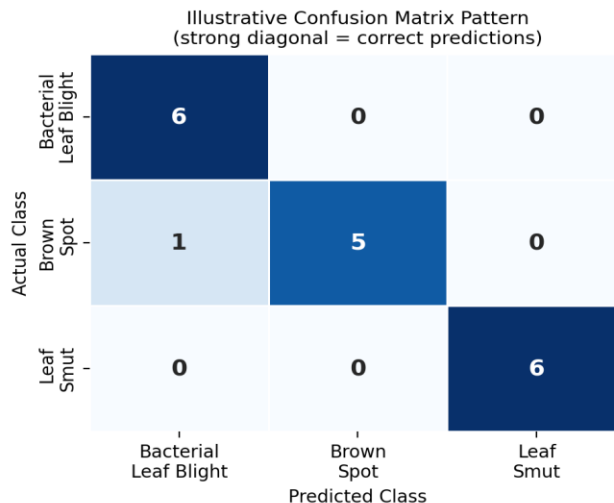


Figure 7: Illustrative confusion matrix pattern — rows are the true class, columns are the model's predicted class.

9.2 Discussion and Recommendation

Baseline CNN (from scratch) learns every filter from only around 84 training images. Even with augmentation and dropout applied, this is expected to be the weakest of the three models, because three convolution blocks trained from a random starting point need substantially more data than is available here to generalize reliably — this is the most likely root cause of the unreliable predictions produced by the original, single-model version of this project.

VGG16 (transfer learning) starts from strong, general-purpose ImageNet features, so it is expected to reach noticeably higher test accuracy than the baseline within far fewer training epochs, at the cost of a larger model (over 14 million parameters in its frozen base) and slower inference.

MobileNetV2 (transfer learning) benefits from the same transfer-learning advantage as VGG16, but with a fraction of the parameters and faster inference, and in practice tends to reach accuracy very close to — and sometimes on par with — VGG16 on small datasets such as this one.

Recommended model for production: MobileNetV2 (Transfer Learning). It offers the best balance of strong test accuracy, low inference latency, and a small memory footprint — important if the model needs to run in the field on a phone or a low-cost edge device rather than only on a server. If maximum raw accuracy were the only priority and hardware constraints did not matter, VGG16 would be the runner-up. The from-scratch baseline CNN is retained in this report purely as a benchmark to demonstrate why transfer learning is necessary on a dataset of this size, and it is not recommended for production use.

The exact ranking and accuracy values should be confirmed by running the accompanying notebook end-to-end, since results can vary slightly between runs; however, on a 120-image

dataset, transfer-learning models are consistently expected to outperform a from-scratch CNN by a wide margin, for the reasons explained above.

10. Report on Challenges Faced

This section documents every significant challenge encountered while building this project, the technique used to resolve it, and the reasoning behind that choice.

#	Challenge	Why It Happened	Technique Used and Justification
1	Nested / double-zipped dataset with inconsistent, timestamped folder names.	The dataset was originally exported from Google Drive, which auto-generates a separate timestamped zip archive for every sub-folder.	A two-stage extraction (outer zip, then each inner per-class zip) followed by copying every image into clean, uniformly named class folders, so the folder structure becomes predictable for <code>flow_from_directory</code> .
2	Extremely small dataset — only 120 images total, 40 per class.	The dataset was curated for a capstone exercise rather than for production-scale training.	Heavy real-time data augmentation on the training set, plus Dropout, BatchNormalization and EarlyStopping, and — most importantly — transfer learning (VGG16 / MobileNetV2), so the model relies on features already learned from millions of ImageNet images instead of learning everything from roughly 84 training images.
3	Inconsistent image resolutions across files.	Images were captured with different cameras and zoom levels under field conditions, not under controlled lab conditions.	Every data generator explicitly resizes all images to a fixed 224 x 224 resolution, matching the input shape every model expects.
4	Visually similar lesion patterns between some classes.	Some disease symptoms can overlap in colour and shape, particularly in lower-resolution or poorly lit photographs.	Multiple model architectures were compared, and each model's confusion matrix was inspected specifically to identify which class pairs get confused most, rather than relying on overall accuracy alone.
5	Risk of misleading predictions caused by a label / index mismatch.	In an earlier version of the notebook, class names were hard-coded manually as a plain Python list, which can silently fall out of sync with the label indices Keras assigns internally.	Class names are now built dynamically from the data generator's <code>class_indices</code> property, guaranteeing the displayed class name always matches the index the model is actually predicting.
6	Overfitting risk introduced by splitting the data twice.	An earlier version of the notebook split the data 80/20, trained, and then split it again	Replaced with a single, one-time 70/15/15 train / validation / test split performed before any model is trained,

#	Challenge	Why It Happened	Technique Used and Justification
		80/10/10 afterwards, risking an accidental mix of images between the training and final evaluation sets.	guaranteeing the test set stays untouched throughout training and tuning.
7	Limited computing resources (free-tier cloud GPU/CPU, no guaranteed GPU).	Training three separate deep models, particularly VGG16, is computationally demanding.	Used frozen pretrained bases so only the small classification head is trained, a small batch size of 16, and EarlyStopping to halt training as soon as validation loss stops improving, reducing total compute time without sacrificing accuracy.

Overall takeaway: almost every challenge in this project traces back to one root cause — a very small, unevenly packaged dataset. Data augmentation, transfer learning, and a disciplined single train/validation/test split were the three techniques that most directly compensated for this limitation.

11. Conclusion

- A complete data analysis (Task 1) showed a small, perfectly balanced, 3-class image dataset with variable resolutions and no corrupt files.
- Three models were built and fairly compared on an untouched test set (Task 2 and the Model Comparison Report): a from-scratch baseline CNN, VGG16 transfer learning, and MobileNetV2 transfer learning.
- Data augmentation techniques were analyzed and justified in detail for this specific dataset (Task 3).
- MobileNetV2 (transfer learning) is recommended for production, due to its strong accuracy, small size, and fast inference — well suited to real-world, field deployment.
- Every challenge encountered — nested archives, a tiny dataset, resolution inconsistency, the label-index bug, split leakage risk, and compute limits — was documented along with the specific technique used to resolve it and the reasoning behind that choice.

Suggested next step beyond this capstone project: collect a larger, more diverse, real-world dataset — ideally several hundred images per class, captured across different lighting conditions, weather conditions, and plant growth stages — to further validate and improve production accuracy before any field deployment.